

Spigot 脚本扩展：JavaScript、Lua 与 Python

本文说明 HuHoBotPenguin 的 Spigot / Paper 适配器如何加载脚本插件。一个插件是 addons/ 下的一个目录，里面放 main.js、main.lua 或 main.py，不是单个文件。

- 适用版本：1.14.0
- 插件名：HuHoBotPenguin
- 加载器：server/Spigot/src/main/java/cn/huohuas001/huhobotPenguin/spigot/scripting/
- Lua 的接线方式移植自 BirdLibraryApi (Apache-2.0)
- JavaScript 与 Python 的上下文建在独立引擎 jar 里，主插件只通过反射调用桥

三种语言拿到的全局对象相同。脚本之间不共享变量。

1. addon 模块

settings.gradle.kts 里的脚本引擎是下面两个。它们不是 Bukkit 插件：没有 plugin.yml，也不注册命令。两个 jar 只含 Graal 运行时和桥，不含 Nukkit 代码，所以另一边的服务端也能把同一对 jar 放进自己的 engines/。

模块	是什么	产物与位置
:addon-GraalJs	GraalJS 24.1.2, 桥 cn.huohuas001.huhobot.graaljs.GraalJsBridge	HuHoBot-Engine-GraalJs-<版本>.jar, 放到 engines/
:addon-GraalPy	GraalPy 24.1.2 (Python 3), 桥 GraalPyBridge	HuHoBot-Engine-GraalPy-<版本>.jar, 放到 engines/
主 jar	LuaJ 3.0.1	已打进 HuHoBot-Penguin-Spigot-<版本>.jar

```
./gradlew :addon-GraalJs:shadowJar :addon-GraalPy:shadowJar
# addon/GraalJs/build/libs/HuHoBot-Engine-GraalJs-<版本>.jar
# addon/GraalPy/build/libs/HuHoBot-Engine-GraalPy-<版本>.jar
```

GraalJS 解压后约 37 MB，GraalPy 约 180 MB。两个 jar 各带一份 polyglot / Truffle 运行时，互不依赖，所以只用其中一种时不必放另一个。构建时直接依赖语言实现 (js-language、python-language)，不用 *-community 那个 POM：它的真正引擎是 runtime scope，Gradle 不传递，shadowJar 会是空壳。

engines/ 在插件启动时创建。第一次加载 .js 或 .py 时，目录里所有 *.jar 挂到同一个 URLClassLoader (父加载器是插件自己的)，结果被缓存。GraalVM 24.1 从当前线程的 context classloader 发现语言，Engine.newBuilder / Context.newBuilder 又没有 classloader 参数，所以建引擎期间线程加载器会被临时换成引擎 jar 的，调用结束再换回来。语言发现本身在引擎 jar 里的桥接类上完成，主插件编译期不依赖 Graal。

不放对应的 jar，插件正常启动，该语言的脚本加载时报「未安装脚本引擎」，其它语言不受影响。

入口	引擎与位置	全局对象
main.lua	LuaJ 3.0.1, 在主 jar	Bird、Bukkit、server、plugin、config、kv、DATA_DIR
main.js	GraalJS 24.1.2, engines/HuHoBot-Engine-GraalJs-*.jar	同上

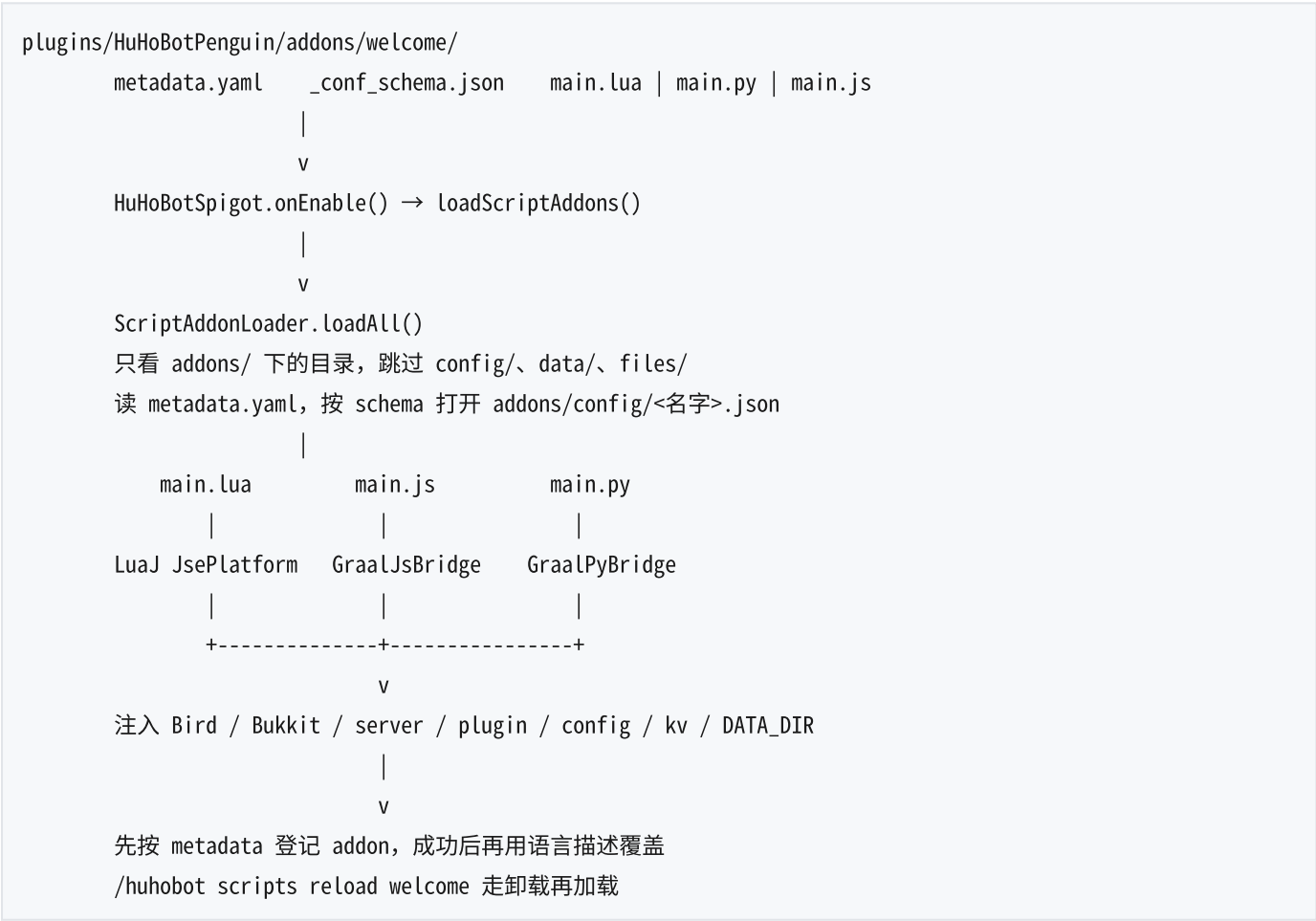
入口	引擎与位置	全局对象
main.py	GraalPy 24.1.2 (Python 3), engines/HuHoBot-Engine-GraalPy-*.jar	同上

名字	是什么	用来做什么
Bird	BirdScriptApi 实例	事件、命令、定时器、玩家、方块、HTTP、QQ 机器人命令。优先用它
Bukkit	org.bukkit.Bukkit 这个类	JS 用 Java.type 不需要它；Lua 直接 Bukkit:getOnlinePlayers()
server	当前 org.bukkit.Server	已经是对象，直接调用
plugin	HuHoBotSpigot 插件实例	一般不需要

Bird 是每个脚本各一份。脚本重载时，这一份登记过的 Bukkit 事件、动态命令、定时任务和 QQ 自定义命令会一起卸掉。

自定义事件总线（Bird.on / Bird.emit）是唯一的跨脚本通道，而且它是进程内静态的：一个脚本 emit，其它脚本用同一个名字 on 就能收到。

2. 框架



卸载（/huhobot scripts reload，或插件 onDisable）的顺序是：

1. Bird.unregisterAll()：取消该脚本的 Bukkit 事件、动态命令、定时任务、QQ 自定义命令。
2. 调用引擎对象的 close()。Lua 的 Globals 没有需要关闭的资源；JS 与 Python 会关掉各自的 polyglot Context。

3. 从已加载表里移除。只重载一个目录时，其它插件不受影响。

3. 加载流程

3.1 何时触发

HuHoBotSpigot.onEnable() 末尾调用 loadScriptAddons()：

1. new ScriptAddonLoader(this)。plugins/HuHoBotPenguin/addons/ 不存在就创建它，engines/ 同样。
2. loadAll()。列出 addons/ 下的目录（跳过 config/、data/、files/），按目录名排序后逐个加载。根上直接放的 .js / .lua / .py 不加载，控制台提示挪进目录。
3. 失败的插件被跳过，插件继续运行。

一个目录里放：

```
addons/welcome/
├── metadata.yaml      name / version / author / description / entry
├── _conf_schema.json  配置声明，可省略
├── requirements.txt   依赖声明，只预检不安装，可省略
└── main.lua          入口。entry 没写时按 main.lua、main.py、main.js 找
```

metadata.yaml 只认顶层 key: value。缺省时名字和作者用目录名，版本 1.0.0。配置实际写在 addons/config/<名字>.json，和代码分开；"_enabled": false 就跳过这个插件。大文件放 addons/data/<名字>/（脚本里的 DATA_DIR），键值放同目录的 kv.properties。

三种语言都多注入三个全局：config、kv、DATA_DIR。Lua 用冒号（config.getString("token")、kv.put("k", "v")），JS 和 Python 用点。config 有 get、getString、getList、set、remove、all；kv 有 get、put、delete、all。

3.2 登记时机

脚本在执行期间就会调用 registerBotCommand，而它要求 addon 已经登记。所以加载器先按 metadata 登记一份，文件跑成功后再用语言描述覆盖：JavaScript script addon / Lua script addon / Python script addon。metadata 里写了描述就用描述。脚本自己再调 Bird.registerAddon(...) 可以改掉元数据。

3.3 .lua

loadLuaScript()：

1. JsePlatform.standardGlobals() 建一套 Lua 全局环境，包含标准库（string、table、math、io、os）。
2. 四个全局都用 CoerceJavaToLua.coerce(...) 包成 Lua 值。之后用冒号调用：Bird.log("hi")。
3. globals.load(输入流，文件名，"t"，globals) 编译，chunk.call() 执行。"t" 表示只允许文本模式。
4. LuaError 取第一行作为错误信息，通常已包含 文件名:行号。

3.4 .js

loadJsScript()：

1. 从 engines/ 反射加载 cn.huohuas001.huhobot.graaljs.GraalJsBridge，调用 open()。找不到 jar 就返回失败。
2. open() 在引擎 jar 的 classloader 上建 polyglot 上下文，选项是：

| 选项 | 作用 | |-----|-----| | allowAllAccess(true) | 允许脚本访问宿主，包括 IO | | HostAccess.ALL, 但 JavaScript 函数保持为原始 Value | 回调能按宿主方法声明的函数式接口适配，而不是被一律映射成 java.util.function.Function | | allowHostClassLookup(s -> true) | Java.type("任意类") 都放行 | | js.nashorn-compat = true | 兼容 Nashorn 写法 (Java.type) |

1. bind 注入 Bird、Bukkit、server、plugin、config、kv、DATA_DIR，再 eval(文件)。执行的是文件顶层，没有 onEnable。
2. 语法或运行错误取第一行记一条日志，并关掉已经打开的上下文。

GraalJsBridge.adapt 把脚本函数适配成宿主声明的接口 (EventCallback、CommandCallback、TabCompleteCallback、Runnable、Consumer)。返回 List 的回调 (Tab 补全) 会转成 Java List。

3.5 .py

loadPythonScript():

1. 第一次遇到 .py 时反射调用 GraalPyBridge.createEngine(), 得到一个共享的 org.graalvm.polyglot.Engine, 并确认它真的提供 python 语言。这个 Engine 被缓存。
2. 每个脚本调用 GraalPyBridge.open(engine) 建自己的 Context (allowAllAccess(true))。脚本之间不共享全局变量，但共用同一个 Engine。
3. 同样注入 Bird、Bukkit、server、plugin、config、kv、DATA_DIR。requirements.txt 里的模块会先 import 一次，缺了只记警告，然后 eval(文件)。顶层即执行。
4. 脚本错误被包成引擎 jar 里的 GraalPyBridge.Failure (主插件 classpath 上没有 PolyglotException)。line() 有源码行号时，日志会带 (line N)。

GraalPy 是 Python 3。Nukkit 侧的 .py 也是 GraalPy，但桥和全局对象不同 (那边是 api，这边是 Bird)，两边的脚本不能互相拷。

3.6 重载

命令	行为
/huohobot scripts reload	卸载全部再加载全部
/huohobot scripts reload welcome	只重载 addons/welcome/，目录名大小写不敏感

命令挂在已有的 /huohobot 下，权限是 huohobot.command (默认 OP)。补全会列出 addons/ 下的目录名。

3.7 失败、重名和禁用

- 入口编译或执行失败：控制台一条 [名字] ... error。加载前已经 registerAddon 的那份会被 unregisterAddon 撤掉，不会在 QQ 菜单里留下一个空插件。其它目录继续加载。
- 两个目录的 metadata.yaml 写成同一个 name：后一个被跳过，日志写「插件名已被另一个目录使用」。重载键是目录名，登记名是 metadata 的 name，两者必须一一对应。
- 脚本跑起来之后抛错 (事件、定时任务、命令回调)：Bird 捕获后写警告，不会把服务器打崩。
- addons/config/<名字>.json 里 "_enabled": false：这个目录不加载，日志记「已禁用」。
- schema 里某个键的 default 是 null 或没写：这个键不进配置表，config.get 返回 null，不会让插件加载失败。
- 配置或 kv 写盘失败：stderr 打 [script-config] 或 [script-kv]，不再静默丢掉。
- 插件 onDisable：unloadAll(), 每个脚本走 Bird.unregisterAll(), 再关掉对应的引擎上下文。

3.8 重载时卸掉什么

/huhobot scripts reload welcome 对这一份 Bird 做：

1. 反注册它登记过的 Bukkit 事件。
2. 从命令表摘掉 onCommand 注册的动态命令。
3. 取消它创建的全部定时任务。
4. 删掉它登记的 QQ 群命令。
5. unregisterAddon，再按同样的流程重新加载这个目录。

其它目录不受影响。Lua 的 Globals 没有要关的资源；JS 和 Python 会关掉自己的 polyglot Context。Spigot 的 GraalPy Engine 是所有 .py 共享的一个，重载单个插件不会把它关掉。

4. 暴露的桥：Bird

Bird 的类型是 BirdScriptApi。JS 和 Python 用点调用，Lua 用冒号调用。参数个数少于重载的会命中更短的那个重载。

4.1 事件

```
Bird.onEvent(类全名, 回调)
Bird.onEvent(类全名, 优先级, 回调)    优先级: LOWEST LOW NORMAL HIGH HIGHEST MONITOR
```

类名必须是**全限定名**，例如 org.bukkit.event.player.PlayerJoinEvent。找不到类只记一条警告，不会中断脚本后面的代码。

回调收到 Bukkit 的 Event。JS / Python 里直接 event.getPlayer()；Lua 里是 event:getPlayer()。

优先级写错时退回 NORMAL。监听器不是 ignore-cancelled：被别的插件取消的事件仍会送到脚本。

4.2 游戏内命令

```
Bird.onCommand(名字, 回调)
Bird.onCommand(名字, 权限, 回调)
Bird.onCommand(名字, 回调, Tab补全回调)
Bird.onCommand(名字, 权限, 回调, Tab补全回调)
```

回调签名是 (sender, label, args)，对应 Bukkit 的 CommandSender、命令标签、String[]。Tab 补全回调签名相同，返回 List<String>；JS 返回数组，Lua 返回 table，Python 返回 list。

命令通过反射拿到的 CommandMap 注册，所以**不需要改 plugin.yml**。设了权限而调用者没有时，调用者收到 You don't have permission to use this command。

4.3 定时任务

方法	含义	返回
runTask(fn)	下一拍在主线程执行	taskId
runTaskLater(fn, ticks)	延迟 ticks 拍，20 拍 = 1 秒	taskId
runTaskTimer(fn, delay, period)	延迟后按周期重复	taskId

方法	含义	返回
runTaskAsync(fn)	异步线程执行一次	taskId
runTaskTimerAsync(fn, delay, period)	异步周期任务	taskId
cancelTask(taskId)	取消	
sleep(秒)	暂停当前线程	见下
waitTicks(ticks)	暂停 ticks * 50 毫秒	见下

sleep / waitTicks 在主线程上调用会被拒绝并记一条警告，因为它会卡住整个服务器。把它包在 runTaskAsync 里。脚本卸载时，它创建的全部任务会被取消。

4.4 玩家与消息

方法	作用
getPlayer(名字) / getPlayerExact(名字)	按名字找在线玩家，找不到返回 null
getOnlinePlayers()	在线玩家列表
getPlayerNames()	在线玩家名列表
getOnlineCount() / getMaxPlayers() / isOnline(名字)	人数
tell(玩家, 文本) / tell(玩家名, 文本)	给一个人发消息，& 颜色码有效；按名字的重载找不到人时返回 false
broadcast(文本) / broadcast(文本, 权限)	广播；第二个重载只发给有该权限的人
broadcastChat(...)	与 broadcast 相同
colorize(文本) / stripColor(文本)	&a 与去色
sendTitle(玩家, 标题, 副标题)	三个时间参数可省略
broadcastTitle(...)	同上，可再加一个权限参数
sendActionBar(玩家, 文本) / broadcastActionBar(...)	退化为普通消息（Spigot 1.16 没有统一的 action bar API）
getHealth / setHealth / getFood / setFood	血量、饥饿
giveExp(玩家, 数量)	经验
setGameMode(玩家, "SURVIVAL") / getGameMode(玩家)	游戏模式，非法值返回 false
addPotionEffect(玩家, 效果名, 秒, 等级) / removePotionEffect / hasPotionEffect	药水效果，名字按 Bukkit 枚举解析
kick(玩家, 原因)	踢出
getDistance(玩家A, 玩家B)	不同世界返回 -1
getPing(玩家) / getPlayerUUID(玩家)	延迟、UUID 字符串
setDisplayname / getDisplayName	显示名
getNearbyPlayers(玩家, 半径)	同一世界、半径内的其他玩家
hasCooldown(键, 玩家) / setCooldown(键, 玩家, 秒)	冷却按「脚本名 + 键 + UUID」隔离
getCooldownRemaining(键, 玩家) / clearCooldown(键, 玩家)	剩余秒数、清除

4.5 世界与方块

方法	作用
<code>getWorldNames()</code>	已加载世界名
<code>getBlock(世界, x, y, z) / getBlockType(...)</code>	拿方块 / 拿类型名
<code>setBlockType(世界, x, y, z, "STONE")</code>	失败（世界或材料不存在）返回 false
<code>isBlockType(...)</code>	比较
<code>getTargetBlock(玩家) / getTargetBlock(玩家, 距离)</code>	准星指着的方块
<code>getBlockPlayerIsOn(玩家)</code>	脚下那一格
<code>getWorldTime(世界) / setWorldTime(世界, ticks)</code>	世界时间
<code>setWeather(世界, true) / isStorming(世界) / isNight(世界)</code>	天气；isNight 为 13000 到 23000 拍
<code>strikeLightning(世界, x, y, z)</code>	真雷
<code>strikeLightning(世界, x, y, z, true)</code>	只播放效果，不造成伤害
<code>teleport(玩家, x, y, z) / teleport(玩家, x, y, z, 世界)</code>	传送

材料名、声音名、粒子名、药水效果名都按 Bukkit 枚举名解析，大小写不敏感。非法值记一条警告并返回，不抛异常。

4.6 物品、声音、粒子

```
Bird.giveItem(玩家, 材料名)
Bird.giveItem(玩家, 材料名, 数量)
Bird.giveItem(玩家, 材料名, 数量, 显示名)
Bird.giveItem(玩家, 材料名, 数量, 显示名, lore列表)
```

背包满时多余的掉在脚下。`hasItem(玩家, 材料, 数量)`、`removeItem(玩家, 材料, 数量)` 按材料统计，数量不足时 `removeItem` 返回 false 且不删除。`getItemInHand(玩家)` 返回主手材料名。`clearInventory(玩家)` 清空背包。

`playSound(玩家, 声音名)`、`playSound(玩家, 声音名, 音量, 音高)`、`broadcastSound(声音名, 音量, 音高)`、`spawnParticle(玩家, 粒子名, 数量)`。

4.7 数据与文件

`setData(键, 值) / getData(键) / getData(键, 默认值) / removeData(键) / getDataKeys() / saveData()`

存在 `plugins/HuHoBotPenguin/addons/data/<脚本名>.properties`，脚本名是文件名去掉扩展名。`setData` 会立刻写盘。

`saveFile(文件名, 内容) / readFile(文件名) / fileExists / deleteFile / listFiles()`

只允许写到 `addons/files/<脚本名>/` 里面。路径逃逸 (`../`) 会被拒绝。下列扩展名一律拒绝：`exe bat cmd dll so sh bash jar php ps1 vbs ...`。

4.8 HTTP 与 Discord

```
Bird.fetch(url, 回调)
Bird.fetch(url, 方法, 回调)
Bird.fetch(url, 方法, body, 回调)
Bird.fetch(url, 方法, body, headers表, 回调)
```

请求在异步线程发，回调切回主线程执行。回调收到 `HttpResult: status、body、ok` (ok 为 2xx)。连接失败时 status 为 0，body 是错误文本。超时 10 秒。有 body 且没给 Content-Type 时默认 application/json。

`sendDiscordWebhook(url, 文本)`、`sendDiscordWebhook(url, 文本, 用户名, 头像url)`、`sendDiscordEmbed(url, 标题, 描述, "#3498db")` 是对 `fetch` 的封装。

4.9 脚本之间的事件

```
Bird.on(名字, 回调)      回调收到一个对象
Bird.emit(名字, 数据)
```

同名的监听在 `emit` 时同步调用。适合一个脚本通知另一个脚本，不要用它代替 Bukkit 事件。

4.10 HuHoBot: QQ 群命令

```
Bird.addonName()          文件名去掉扩展名
Bird.registerAddon(名字)
Bird.registerAddon(名字, 版本, 描述)
Bird.registerAddon(名字, 版本, 描述, 作者)      加载器已经登记过，用来改元数据
Bird.registerBotCommand(addon名, key, 命令模板)  权限公开，推送到 QQ 菜单
Bird.registerBotCommand(addon名, key, 命令模板, 权限, 是否推送到QQ菜单)
Bird.unregisterBotCommand(key)
Bird.sendBotText(文本)      发到所有已配置的 QQ 群
Bird.sendBotMarkdown(markdown)
```

`registerBotCommand` 的第一个参数必须是**已经登记的 addon 名**，也就是 `Bird.addonName()` 的返回值。命令模板 是一条服务器命令，支持与配置文件自定义命令相同的占位符：`{params}`、`{group}`、`{user}`、`{name}`、`{nickname}`、`{0}`、`{1}`。权限 0 为公开，大于 0 仅管理员。

脚本重载时，这一份 `Bird` 登记过的 key 会被删掉。

4.11 服务器命令

```
Bird.runCommand("say hi")      以控制台身份，下一拍执行
Bird.runCommandAs(玩家, "spawn") 以该玩家身份
```

两条都会去掉开头的 `/`，并切回主线程执行。

4.12 其它

`log(消息) / warn(消息)` 写到服务器日志，前缀是 [文件名]。`random(min, max)` 含两端。`formatTime(秒)` 返回 HH:MM:SS。`getServer()` 返回 Bukkit Server。

5. JavaScript 示例

入口：`plugins/HuHoBotPenguin/addons/welcome/main.js`

需要先把 `HuHoBot-Engine-GraalJs-<版本>.jar` 放到 `engines/`。

```

var PlayerJoinEvent = "org.bukkit.event.player.PlayerJoinEvent";

Bird.onEvent(PlayerJoinEvent, function (event) {
    var player = event.getPlayer();
    Bird.tell(player, "&a欢迎来到服务器, &f" + player.getName());
    Bird.playSound(player, "ENTITY_EXPERIENCE_ORB_PICKUP", 1, 1);
});

Bird.onCommand("hello", "huhobot.command", function (sender, label, args) {
    Bird.tell(sender, "&e你好, " + sender.getName());
}, function (sender, alias, args) {
    return ["world", "me"];
});

Bird.registerBotCommand(Bird.addonName(), "欢迎", "say {name} 说: {params}");

Bird.runTaskTimer(function () {
    Bird.broadcast("&7当前在线 &f" + Bird.getOnlineCount() + "&7 人");
}, 20, 20 * 30);

Bird.fetch("https://example.com/motd.txt", function (result) {
    if (result.ok) Bird.log("motd 长度 " + result.body.length);
});

var GameMode = Java.type("org.bukkit.GameMode");
var player = Bird.getPlayer("Steve");
if (player != null) player.setGameMode(GameMode.CREATIVE);

```

加载失败时控制台会看到带原因的 `[welcome.js] Failed to load: ...`。这条脚本不会作为可用 addon 留下，同目录其它脚本照常加载。

6. Lua 示例

入口: `plugins/HuHoBotPenguin/addons/welcome/main.lua`

Lua 不需要引擎 jar。Java 方法都用冒号调用。

```

Bird:onEvent("org.bukkit.event.player.PlayerJoinEvent", function(event)
    local player = event:getPlayer()
    Bird:tell(player, "&a欢迎, &f" .. player:getName())
    Bird:playSound(player, "ENTITY_EXPERIENCE_ORB_PICKUP", 1, 1)
end)

Bird:onCommand("hello", "huhobot.command", function(sender, label, args)
    Bird:tell(sender, "&e你好, " .. sender:getName())
end, function(sender, alias, args)
    return {"world", "me"}

```

```

end)

Bird:registerBotCommand(Bird:addonName(), "欢迎", "say {name} 说: {params}")

Bird:runTaskTimer(function()
    Bird:broadcast("&7当前在线 &f" .. Bird:getOnlineCount() .. "&7 人")
end, 20, 20 * 30)

Bird:fetch("https://example.com/motd.txt", function(result)
    if result.ok then
        Bird:log("motd 长度 " .. #result.body)
    end
end)

local players = Bukkit:getOnlinePlayers()
Bird:log("Bukkit 报告在线 " .. players:size())

```

- `Bird:getOnlineCount()` 而不是 `Bird.getOnlineCount()`。点号调用会丢掉 `self`，参数全部错位。
- LuaJ 的数组下标从 1 开始，但 Java 的 `String[]` 仍然从 0 开始：`args[0]` 才是第一个参数。
- `return {"world", "me"}` 能被转成 `List<String>`，因为 LuaJ 会把顺序表封成 Java List。

7. Python 示例

入口：`plugins/HuHoBotPenguin/addons/welcome/main.py`

需要 `HuHoBot-Engine-GraalPy-<版本>.jar`。这是 Python 3 (GraalPy)，用点和普通函数，没有 Nukkit 那套 `api.register_event`。

```

def on_join(event):
    player = event.getPlayer()
    Bird.tell(player, "&a欢迎, &f" + player.getName())
    Bird.playSound(player, "ENTITY_EXPERIENCE_ORB_PICKUP", 1, 1)

def on_hello(sender, label, args):
    Bird.tell(sender, "&e你好, " + sender.getName())

Bird.onEvent("org.bukkit.event.player.PlayerJoinEvent", on_join)
Bird.onCommand("hello", "huhobot.command", on_hello)
Bird.registerBotCommand(Bird.addonName(), "欢迎", "say {name} 说: {params}")

def tick():
    Bird.broadcast("&7当前在线 &f" + str(Bird.getOnlineCount()) + "&7 人")

Bird.runTaskTimer(tick, 20, 20 * 30)

def on_motd(result):
    if result.ok:

```

```
Bird.log("motd 长度 " + str(len(result.body)))
```

```
Bird.fetch("https://example.com/motd.txt", on_motd)
```

语法错误会带行号，例如 `[welcome.py] Python error: ... (line 4)`。

8. 一个脚本的生命周期

服务器启动

- └─ HuHoBotSpigot.onEnable
 - └─ ScriptAddonLoader.loadAll
 - └─ 先按文件名 registerAddon
 - └─ 建引擎、注入 Bird
 - └─ 执行文件顶层（这里面调用 onEvent / onCommand / registerBotCommand）
 - └─ 成功后再用语言描述覆盖 addon 元数据

玩家进服 / 执行命令 / 定时器到点

- └─ Bukkit 回调进 BirdScriptApi
 - └─ 转进脚本函数；异常被吃掉并写日志

/huhobot scripts reload hello

- └─ 找到已加载的 addons/hello/
 - └─ Bird.unregisterAll() 事件、命令、任务、QQ 命令
 - └─ close()
- └─ 重新 loadScript

服务器关闭

- └─ onDisable → unloadAll → 每个脚本 unregisterAll

脚本没有 onDisable 钩子。卸载只保证注册出去的东西被反注册。

9. 安全边界

这套引擎给脚本的权限很大，是有意的：脚本作者就是服务器管理员。

- JS 的 `Java.type` 可以加载任意类，`allowAllAccess` 打开了文件访问。
- Lua 的 `io`、`os` 标准库是开的。
- Python 的 `allowAllAccess` 同样能访问宿主。
- `Bird.fetch` 能向任意地址发请求。
- `Bird.runCommand` 以控制台身份执行命令。

不要把 `addons/` 目录暴露给不信任的人。`Bird.saveFile` 的路径限制只挡住 `Bird` 自己的文件 API，挡不住 `Java.type("java.io.File")`、Lua 的 `io.open`，或 Python 里的 `open`。

10. 目录与命令速查

plugins/HuHoBotPenguin/

```
|—— addons/
|   |—— welcome/          一个目录一个插件
|   |   |—— metadata.yaml
|   |   |—— _conf_schema.json
|   |   |—— main.js        或 main.lua / main.py
|   |—— config/           schema 生成的实际配置
|   |   |—— welcome.json
|   |—— data/             DATA_DIR 与 kv.properties; Bird.setData 也在这
|   |   |—— welcome/
|   |—— files/            Bird.saveFile 的根
|   |   |—— welcome/
|—— engines/              JS 与 Python 需要, Lua 不需要
    |—— HuHoBot-Engine-GraalJs-<版本>.jar
    |—— HuHoBot-Engine-GraalPy-<版本>.jar
```

/huhobot scripts reload

/huhobot scripts reload welcome